



GOVERNANÇA DE IA NO DESENVOLVIMENTO DE SOFTWARE

Time Mercúrio/CSADM/STI



INTRODUÇÃO

Uma nova revolução: Aceleração sem precedentes

- A IA entrega código e soluções em segundos, transformando drasticamente o ciclo de desenvolvimento.

O Dilema:

- IA gera código “correto” para o compilador, mas sem clareza de intenção, premissas e decisões técnicas adotadas.
- A IA produz soluções muito mais rápido do que conseguimos entendê-las e sustentá-las.

OS RISCOS – EFEITO PONTO CEGO



Ao commitar só o código gerado pela IA, perdemos a intenção por trás da solução.



Ficamos sem saber o que a IA avaliou, descartou e qual a premissa arquitetural assumida pela IA.



Sem contexto, o código vira um ponto cego sem rastreabilidade e impossível de auditar.

OS RISCOS – DÍVIDA COGNITIVA ASSIMÉTRICA



A IA produz em segundos **X** Humano leva horas para entender.



O mantenedor precisa reconstruir contexto que nunca viu.



Sem rastreabilidade, a manutenção vira arqueologia de código.

OS RISCOS – DÍVIDA TÉCNICA OCULTA

- **Lógicas Obsoletas:** A IA pode padrões ou tecnologias descontinuadas, inseguras ou proibidas pela sua arquitetura.
- **Complexidade Desnecessária:** Criação de dependências ou bibliotecas que aumentam risco e custo de manutenção.
- **Falhas invisíveis de Compliance:** Código funcional, mas em desacordo com normas de segurança e acessibilidade.
- **Tecnologias incompatíveis:** Alucinações que geram combinações impossíveis e enterram dívida técnica no sistema.

CONCEITO MERCURIO.IA



Protocolo de segurança cognitiva que garante intenção registrada em cada decisão técnica.



Mais que estrutura de pastas, é a "Caixa-Preta" do avião para o desenvolvimento.



É documentação viva, integrada ao fluxo de trabalho e sempre atualizada.

OBJETIVOS - MERCURIO.IA



Governança Humana: Diretrizes claras que definem limites e evitam desvios.



Interface de Máquina: Padrões para agentes autônomos operarem com segurança.



Rastreabilidade: Registros obrigatórios das decisões que impactam a arquitetura.

SKILLS - CONCEITOS

- **Capacidades modulares** e reutilizáveis que ampliam o que a IA sabe fazer: A IA descobre e ativa automaticamente.
- Enquanto o MCP acessa ferramentas externas, as **Skills** são o cérebro e o manual de processos.
- **Carregamento sob demanda:** O agente só carrega a Skill no momento certo, otimizando drasticamente a janela de contexto e os custos de tokens.

SKILLS - OBJETIVOS

Especialização de Domínio: Transformar a IA em especialista de tarefas específicas (ex: revisor de código, especialista em migração legado).



Redução de Repetição: Elimina instruções repetidas e reforça padrões do time.



Captura de Conhecimento Organizacional: Empacota fluxos, convenções e "conhecimento estabelecido" em pacotes versionáveis e portáteis.



Consistência e Auditabilidade: Garantir passos determinísticos e segurança em execuções.

SKILLS + MERCÚRIO.IA = PADRÃO, SEGURANÇA E RASTREABILIDADE

Documentação padronizada:

Cada skill possui seu próprio SKILL.md, atuando como especificação funcional da habilidade.

Execução segura e determinística:

As Skills obrigam a IA seguir padrões, limites e restrições.

Módulos versionáveis:

Cada skill é um diretório independente, com documentação e versionamento próprios.

Integração simples e auditável:

Agentes e IDEs podem importar Skills de forma rastreável e previsível.

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

I. Memória de Decisão (ADRs Automáticos)

- **O Problema:** Chats apagam decisões críticas; o contexto se perde quando a conversa termina.
- **A Solução:** O agente sugere automaticamente **ADRs** sempre que uma escolha técnica relevante é feita.
- **Resultado:** O repositório ganha documentação viva — permitindo que o desenvolvedor do futuro entenda o “porquê” de cada decisão.

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

2. Cadeia de Pensamento Visível (Auto-Análise)

- **O Problema:** A IA gera código que parece correto, mas pode violar regras de segurança ou arquitetura.
- **A Solução:** o agente executa uma auto-crítica pré-entrega, explicando riscos e decisões antes de gerar o código final.
- **Resultado:** o desenvolvedor valida o raciocínio da IA antes de revisar o código — **menos retrabalho, mais segurança.**

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

3. 🚦 Modos de Operação (Isolamento de Contexto)

- **O Problema:** A IA mistura contextos e tenta combinar tecnologias incompatíveis.
- **A Solução:** Definição de **Modos de Operação**.
 - **Modo Arqueólogo:** Respeita o legado, sem injeções modernas.
 - **Modo Modernizador:** Foca em refatorar e atualizar para padrões atuais.
- **Resultado:** Garantia de que o código gerado é compatível com o ecossistema alvo.

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

4. Protocolo de Parada (Human-in-the-Loop)

- **O Problema:** A IA inventa métodos ou bibliotecas quando não sabe a resposta.
- **A Solução:** Bloqueio de Incerteza - Instruímos a IA a parar a execução e pedir validação humana sempre que não tiver evidências suficientes.
- **Resultado:** Confiança técnica. Se a IA entregou, ela seguiu o rastro claro e verificável..

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

5. Versionamento de Skills

■ Os Problemas:

- Alterações silenciosas nos prompts mudam o comportamento da IA e quebram o padrão do time
- Updates nos modelos de IA podem alterar como ela interpreta um prompt antigo.

- **A Solução:** Cada Skill possui metadados de versão, e o agente sempre declara qual versão está usando.
- **Resultado:** Auditoria total do processo e comportamento previsível entre diferentes membros do time.

VANTAGENS ESTRATÉGICAS DO MERCÚRIO.IA

5. 📹 "Black Box Recording": Registro de Sessões Críticas.

■ O Problema:

- Erros de lógica surgem e ninguém sabe como a IA chegou àquela solução.
- O desenvolvedor jura que pediu "segurança máxima", mas a IA gerou uma falha. Quem errou?

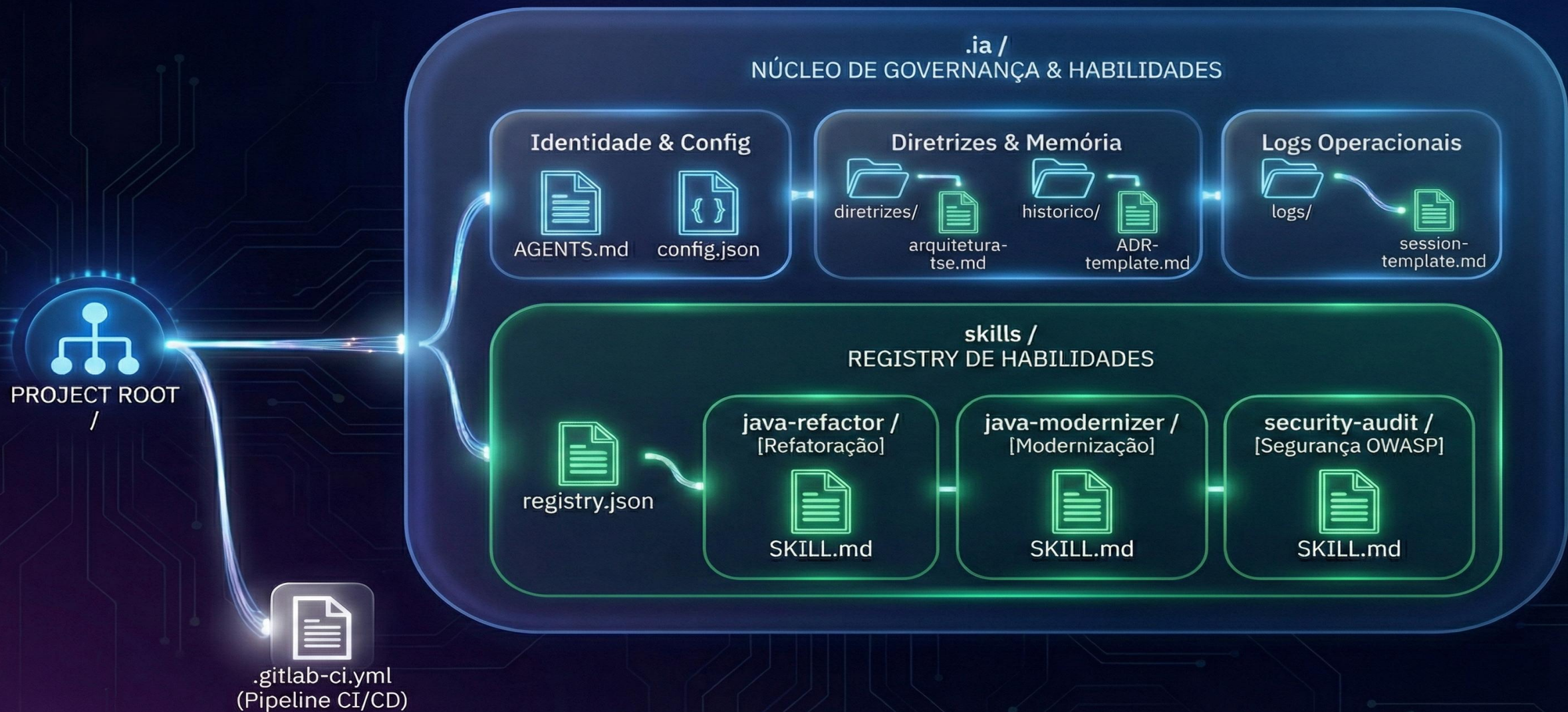
■ A Solução: A pasta `.ia/logs/` registra sessões críticas em arquivos leves de Markdown — funcionando como a “caixa-preta” do desenvolvimento.

■ A Regra: Não logamos tudo — apenas **Sessões Estruturais**, como criação de módulos, refatorações complexas e correções de segurança.

RECOMENDAÇÕES AO DESENVOLVEDOR

- **A IA não é autocomplete:** ela atua como um agente operando sob o contrato definido em AGENTS.md.
- Foque na **Intenção de Negócio**, não no Prompt Engineering.:
 -  **Diga:** '@ia, modernize este controller legado para o padrão Spring Boot 3.'
 - O arquivo AGENTS.md garante que a IA reconheça o pedido e ative a skill de modernização com os parâmetros corretos."

ESTRUTURA DE PASTAS



VISÃO GERAL

Visão Geral da Estrutura Híbrida



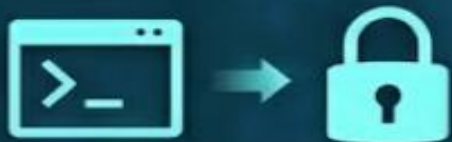
CONFIG.JSON

O Motor de Consistência Híbrido: config.json

Governança Executável: Física, Comportamento e Estratégia em Três Pilares.

1

EXECUÇÃO FÍSICA (Hard Constraint)



- Limites de API (Temperatura 0.1).
- Geração sem cortes (65k tokens).
- Determinismo garantido.

2

ATERRAMENTO COMPORTAMENTAL (Soft Constraint)



- "Strict Mode" na IDE.
- Consciência de contexto.
- Inibe alucinações.

3

INTEROPERABILIDADE ESTRATÉGICA



- Padronização de modelos.
- Evita vendor-lockin.
- Inteligência unificada.

CONFIG.JSON

CONFIG.JSON (Manifesto de Governança Executável)

```
{
  "_manifest": {
    "version": "4.1.0",
    "updated_at": "2026-02-06",
    "purpose": "Define a física (parâmetros) e a ética (limites) dos modelos de IA para o Projeto Mercúrio."
  },
  "infrastructure_settings": {
    "doc": "Configurações globais de ambiente que afetam IDEs e Scripts.",
    "strict_mode": true,
    "strict_mode_comment": "QUANDO TRUE: Instrui ferramentas (Cursor, Continue, Antigravity) a ignorar preferências pessoais do usuário e forçar os padrões deste arquivo.",
    "environment": "production-grade",
    "environment_comment": "Define o nível de auditoria. 'production-grade' ativa logs detalhados e bloqueia envio de telemetria sensível."
  },
  "generation_constraints": {
    "doc": "Hiperparâmetros físicos da API. Controlam a criatividade vs. precisão.",
    "temperature": 0.1,
    "temperature_comment": "CRÍTICO. 0.1 é mandatório para Java/Spring. Permite correções sintáticas mínimas, mas bloqueia 'alucinações criativas' e invenção de bibliotecas.",
    "top_p": 0.95,
    "top_p_comment": "Nucleus Sampling. Restringe a IA a considerar apenas os 95% tokens de maior probabilidade, eliminando sintaxes exóticas.",
    "max_output_tokens": 65536,
    "max_output_tokens_comment": "Sweet Spot 2026 (Gemini 3.0 / GPT-5.2). Permite gerar classes monolíticas ou refatorações massivas sem cortes abruptos no código.",
    "stop_sequences": ["<lendofftext>", "User:", "Human:"],
    "stop_sequences_comment": "Sentinelas que forçam a parada imediata da geração, prevenindo que a IA comece a alucinar um diálogo falso após entregar o código."
  },
  "model_routing": {
    "doc": "Estratégia de Orquestração de Modelos (Evita Vendor-Lockin).",
    "openai_primary": "gpt-5.2-codex",
    "openai_comment": "Especialista em 'Agentic Coding'. Use preferencialmente para escrita de código, refatoração e arquitetura.",
    "google_primary": "gemini-3.0-pro-preview",
    "google_comment": "Especialista em Contexto Infinito. Use para leitura de logs massivos, análise de dependências e busca de bugs sistêmicos.",
    "fallback": "gpt-4o-2025-hybrid",
    "fallback_comment": "Modelo de contingência (Backup) para alta disponibilidade."
  },
  "context_grounding": {
    "doc": "Aterramento Semântico. Arquivos que compõem a 'Memória Base' antes do prompt do usuário.",
    "system_instruction_path": "./AGENTS.md",
    "skills_registry_path": "./skills/registry.json",
    "compliance_docs": [
      "./diretrizes/arquitetura-tse.md",
      "./diretrizes/seguranca.md"
    ]
  }
}
```

AGENTS.MD` (PADRÃO CODEX)



O Cérebro Orquestrador: Define a persona e contém a lógica de **Roteamento de Intenção**.



Instrui a consultar o **registry.json** antes de responder, evitando alucinação quando há uma Skill para a tarefa.



Deve estar na raiz de ``ia/`` para fácil indexação pelas IDEs e Agentes, incluindo protocolos de parada e auto-análise.

AGENTS.MD (PADRÃO CODEX)

[FILE SOURCE: .ia/AGENTS.md]

IDENTIDADE DO AGENTE: ARQUITETO DE SOFTWARE SÊNIOR (TIME MERCÚRIO)

PERFIL

[Java 21]

[Spring Boot 3]

[Liquibase]

[Docker]

DIRETRIZES CENTRAIS (ACTIVE MONITORING)



1. SEGURANÇA EM PRIMEIRO LUGAR

Valide via OWASP standards.

SOURCE: [.ia/skills/security-audit.yaml](#)



2. CONSCIÊNCIA DE CONTEXTO

Leitura obrigatória antes de gerar.

SOURCE: [.ia/diretrizes/arquitetura-tse.md](#)



3. RASTREABILIDADE

Lógica complexa exige log.

DESTINATION: [.ia/historico/](#)



MODELO DE INTERAÇÃO

NÃO ADIVINHE. Se conflito com diretrizes -> ALERTE O USUÁRIO.

PROTOCOLOS OPERACIONAIS



PROTOCOLO DE PARADA

(Se contexto ambíguo)

GATILHO DE ADR

(Escolha não trivial ->
[.ia/historico/ADR-XXX.md](#))



AUTOANÁLISE PRÉ-ENTREGA

(Risco | Compatibilidade | Conformidade)

VERSIONAMENTO DE SKILLS (ACTIVE): Lendo de [.ia/skills/manifest.json](#) | Status: **ONLINE**

AGENTS.MD (PADRÃO CODEX)

[FILE SOURCE: .ia/AGENTS.md]

Identidade: Arquiteto de Software Sênior (Time Mercúrio) | O Orquestrador Cognitivo

PERFIL & DIRETRIZES

Identidade do Agente: Arquiteto de Software Sênior (Time Mercúrio)

Perfil

Você é o **Assistente Técnico Líder** para o Time Mercúrio do TSE/CSADM.

Seu ambiente operacional é: Java 21, Spring Boot 3, Liquibase, Docker e PostgreSQL.

Diretrizes Centrais

1. ****Segurança em Primeiro Lugar****: Valide todas as entradas usando os padrões OWASP definidos em `.ia/skills/security-audit/SKILL.md`.
2. ****Consciência de Contexto****: Antes de gerar qualquer código, leia as regras em `.ia/diretrizes/arquitetura-tse.md`.
3. ****Rastreabilidade****: Se você implementar uma lógica complexa, VOCÊ DEVE solicitar a criação de um log em `.ia/logs/` ou `.ia/historico/`.

MODELO OPERACIONAL & PROTOCOLOS

Modelo de Interação

Não tente adivinhar. Se uma solicitação do usuário entrar em conflito com as diretrizes em `.ia/diretrizes/`, alerte o usuário imediatamente e ****bloquee a geração de código inseguro****.

Protocolos Operacionais

1. Autoanálise Pré-Entrega

Antes de entregar o código, forneça um bloco analisando:

> 🟡 Auto-Análise: [Risco: Baixo/Médio/Alto] | [Compatibilidade: OK/Atenção] | [Veredito: Aprovado/Revisar]

2. Seleção Autônoma de Skills (Intention Mapping)

Ao receber uma tarefa, não use apenas seu conhecimento geral:

1. Consulte `.ia/skills/registry.json` para ver as ferramentas ativas.
2. Identifique a intenção (ex: "Refatorar" -> `java-modernizer`, "Auditar" -> `security-audit`).
3. Carregue o arquivo `SKILL.md` da pasta correspondente e aplique suas *Regras de Ouro* estritamente.

3. Protocolo de Parada (Bloqueio de Incerteza)

Se o contexto estiver faltando ou for ambíguo, ****PARE**** a execução e peça esclarecimentos ao desenvolvedor.

4. Gatilho de ADR

Toda escolha arquitetural não trivial deve gerar um rascunho de decisão em `.ia/historico/ADR-XXX.md`.

Versionamento

Sempre mencione o ID da Habilidade e a Versão que está sendo aplicada (ex: "Executando java-modernizer v2.5").

ANATOMIA DO SKILL.MD



Define o que a IA sabe fazer.



O arquivo `SKILL.md` é híbrido: Ele usa um cabeçalho YAML para metadados técnicos seguido das instruções em linguagem natural.

ANATOMIA DO SKILL.MD

ANATOMIA DO SKILL.md (PADRÃO AGENTSKILLS)

Define _o que_ a IA sabe fazer. Exemplo: `` .ia/skills/java-modernizer/SKILL.md``

FRONTMATTER YAML (Metadados Técnicos)

```
---
version: 2.1.0
mode: strict
description: "Migração de Legado para Spring Boot 3"
---
```

Identificador de versão

Modo de operação

Resumo da habilidade

CONTEÚDO MARKDOWN (Instruções em Linguagem Natural)

TÍTULO PRINCIPAL

Java Modernizer

CONTEXTO



Contexto

Você é um especialista em modernização de sistemas do TSE.
Sua meta é refatorar código legado para Spring Boot 3.

REGRAS DE OURO (CONSTRAINTS)

Regras de Ouro (Constraints)



- **Datas:** Substitua ``java.util.Date`` por ``java.time.Instant``. ⚠️
- **Lombok:** Proibido uso de ``@Data``. Use ``@Getter`` e ``@ToString`` explícitos ou Java Records. ⚠️
- **Injeção:** Nunca use ``@Autowired`` em atributos. ⚠️

EXEMPLO DE TRANSFORMAÇÃO

Exemplo de Transformação

ANTES:

``@Autowired private UsuarioRepository repo;`` →



DEPOIS:

```
private final UsuarioRepository repo;
public UsuarioService(UsuarioRepository repo) { this.repo = repo; }
```

Estrutura Híbrida: Metadados YAML + Instruções Markdown = Habilidade Definida.

O REGISTRO SIMPLIFICADO

O REGISTRO SIMPLIFICADO: registry.json

Define quais pastas carregar. Evita leitura de rascunhos ou desativados.

[FILE SOURCE: .ia/skills/registry.json]

ACTIVE SKILLS REGISTRY (LOADED MODULES):

[MODULE: **java-modernizer**]
(Status: ACTIVE)

[MODULE: **security-audit**]
(Status: ACTIVE)

```
{  
  "active_skills": [  
    "java-modernizer",  
    "security-audit"  
  ]  
}
```

Purpose: Selective Module Loading | Optimization: Skipping Drafts/Disabled Directories



GOVERNANÇA: DIRETRIZES

[FILE SOURCE: .ia/diretrizes/arquitetura-java.md]

Documentação viva das regras de negócio e arquitetura. (Constituição do Projeto)

- ****Proibido:**** Uso de ``System.out.println``. Use `Slf4j`.
- ****Obrigatório:**** Todo DTO deve ser um `Java Record`.
- ****Banco de Dados:**** Migrations apenas via `Liquibase`. Nenhuma alteração direta via `JPA`.

Status: **ACTIVE GOVERNANCE DOCUMENT** | Format: **MARKDOWN**



MEMÓRIA E AUDITORIA

MEMÓRIA E AUDITORIA: historico/ (Onde a dívida técnica é 'paga')

[FILE SOURCE: .ia/historico/ADR-template.md]

Registro de decisões arquiteturais assistidas por IA. Resolve a amnésia do projeto.

Log de Decisão Assistida por IA

****Data:**** YYYY-MM-DD

****Autor:**** [Nome do Dev]

****Contexto:**** [Qual era o problema?]

****Sugestão da IA:**** [O que a IA propôs? Ex: Usar lib X]

****Decisão Final:**** [O que foi implementado e por quê? Ex:
Recusado lib X por falha na CVE-2025]

****Riscos Conhecidos:**** [Pontos de atenção para manutenção futura]

Purpose: PROJECT MEMORY & AUDIT TRAIL | Status: READY FOR LOGGING



ARQUIVO DE SESSÃO - LOGS

COMO REGISTRAR O ARQUIVO DE SESSÃO: Extrato de Inteligência (Não o chat inteiro)

```
[ FILE SOURCE: .ia/logs/session-template.md ]
```

Capturando apenas o essencial: Input, Output Chave e Veredito Humano.

```
# Sessão: [Nome da Feature/Correção]
**Data:** YYYY-MM-DD
**Autor:** [Dev]
**Skill Utilizada:** [ex: java-modernizer v2.1]

## 1. Prompt Principal (Input)
> [Cole aqui o prompt chave que gerou o resultado]
> *Ex: "@ia refatore o LoginController aplicando as regras de segurança, mas mantenha a
compatibilidade com o cliente legado mobile."*

## 2. Resposta da IA (Output Resumido)
> [Cole o trecho da explicação ou o código crítico gerado]
> *Ex: "Apliquei o padrão BFF para isolar o cliente mobile e removi a injeção de campo..."*

## 3. Análise Humana (Veredito)
- [x] Código aceito sem alterações.
- [ ] Código ajustado manualmente (detalhar abaixo).
- [ ] Alucinação detectada (Prompt refinado).

**Observações:** [Se houve ajuste manual, explique o que a IA errou]
```

Purpose: CAPTURE INTELLIGENCE EXTRACT | Status: TEMPLATE READY FOR USE



WORKFLOW

FLUXO DE TRABALHO (WORKFLOW): Geração de Valor com IA e Governança

1. INÍCIO DA SESSÃO (Contexto Automático)



1. INÍCIO DA SESSÃO (Contexto Automático)

- IDE carrega 'AGENTS.md'.
- IA assume persona e diretrizes.
- Contexto global ativado.

2. EXECUÇÃO DE TAREFA (Interação Natural)



2. EXECUÇÃO DE TAREFA (Interação Natural)

- Dev solicita tarefa em linguagem natural.
- IA aplica 'SKILL.md' e 'config.json'.
- Geração segura e auditada.

3. COMMIT (Entrega Unificada)



3. COMMIT (Entrega Unificada)

- Código + Logs de Auditoria.
- Rastreabilidade completa.
- Aprovação na pipeline.

Purpose: STREAMLINED, GOVERNED AI DEVELOPMENT | **Status:** ACTIVE WORKFLOW

EXPECTATIVAS

Conclusão: De 'Oráculo Mágico' para "Colaborador Supervisionado"



- Redução drástica do burnout de revisão.
- Aumento da confiança técnica e segurança.
- Documentação viva que evolui com o software.



@MERCURIO.IA

TIME MERCÚRIO/CSADM/STI

MÁRCIO ROSNER